Parallel Context Compaction for Long-Horizon LLM Agent Serving

Musa Cim, Burak Topcu, Chita Das, Mahmut Kandemir
The Pennsylvania State University
{mtc5693, bvt5283, cxd12, mtk2}@psu.edu

Abstract

Long-horizon LLM agents accumulate growing conversation histories that eventually exceed the model's context window. Context compaction via LLM-based summarization keeps the conversation bounded, but summarization is inherently lossy and the blocking call stalls agent inference for tens of seconds. Moreover, the operator has no fine-grained control over summary volume since prompt instructions are largely ignored, and as context grows, both the amount of output tokens the model produces and the information it retains fluctuate substantially from run to run, making the agent's retained knowledge unpredictable across runs. We introduce ***parallel compaction*** *for long-horizon agentic flows and characterize it against the sequential synchronous baseline across four backbones spanning 8B to 120B parameters, mixing dense and MoE architectures with reasoning and non-reasoning models, on the HotpotQA multi-hop QA and LoCoMo long-context dialogue benchmarks. Parallel compaction gives the operator fine-grained, predictable control over summary volume and enables more targeted prompt engineering per block. At matched compaction decode volume, it reduces end-to-end wall time and improves compaction throughput over the sequential baseline.*

1. Introduction

Large Language Model (LLM) agents are increasingly deployed in long-horizon settings where they process many tasks within a single session, e.g., multi-hop question answering over accumulated evidence, agentic code editing with tool use, and multi-session dialogue. In each setting, the conversation history grows with each interaction, surfacing many practical problems: per-token latency increases with sequence length, the conversation eventually exceeds the model's context window, and reasoning quality degrades well before that window is reached, as the model is forced to attend over an increasingly long, low-signal history. Summarization, the natural remedy, faces its own limits: models struggle to produce summaries of predictable length and content as context grows.

Context compaction periodically summarizes the conversation history into a shorter representation, keeping the context bounded as the agent processes more tasks. Production systems such as Anthropic's Claude Code [2] and OpenAI's Codex [15], along with widely used frameworks such as LangChain [10] and LlamaIndex [12], rely on LLM summarization calls to compress context when approaching window limits. This compaction is typically performed synchronously, and because a summary is inherently shorter than the original, information is always lost: the context is reduced by 90–99% in token count, yet the model decides what to keep and what to drop with no guarantee of consistency across runs. In interactive settings such as CLI coding agents, the information loss is particularly damaging: summarization is so aggressive that the most recent context, which contains the insights the agent has just accumulated, is heavily compressed alongside the rest, forcing the agent to rediscover and re-establish that context in subsequent turns at the cost of extra tokens and time.

Current frontier models support context windows from 100k to 1M tokens (roughly 150–2,500 PDF pages of dense text). Yet feeding this much material into a single inference call yields diminishing returns on most use cases: well-known limitations such as *lost in the middle* [11] and *context rot* [6] show that LLMs struggle to attend over long, low-signal histories, and the same limitations apply during summarization; for example, a model asked to compress a 96k-token context attends poorly over the full history, producing short and inconsistent summaries. The summary volume, information retention, and latency cost of context compaction under long-horizon multi-turn agentic flows have not been systematically characterized, neither for the naive sequential baseline nor for parallel approaches. Our contributions are:

1) **Characterization of compaction behavior.** We show empirically that summarization output volume is essentially input- and prompt-invariant across four backbones spanning 8B to 120B parameters. As input grows from 2k to 96k tokens, output grows by only $\sim 3\times$, and switching the instruction from *concise* to *very detailed* barely moves the output. Moreover, both summary length and content become increasingly unstable at longer contexts, making information loss unpredictable for the agent.
2) **Parallel compaction design and characterization.** We introduce parallel compaction, a block-based design that gives the operator direct, fine-grained control over summary volume through block count, and characterize it against the sequential baseline on HotpotQA and LoCoMo across four backbones, reporting end-to-end wall time, compaction throughput, and summary volume across a block-size sweep from 16k down to 2k tokens.

2. Background and Motivation

2.1. Long-Horizon LLM Agents

We consider an LLM agent that processes a sequence of T tasks $(q_1, q_2, \ldots, q_T)$ within a single conversation session. At step t, the agent receives task q_t appended to the accumulated conversation history $\mathcal{H}_t = [s, u_1, a_1, \ldots, u_{t-1}, a_{t-1}]$, where s is the system prompt, u_i is the ith user message including supporting context, and a_i is the corresponding model response. As more tasks are processed, the growing context length raises per-token decode cost.

2.2. Context Compaction

When the context length exceeds a threshold τ, compaction summarizes the conversation history into a shorter representation:

$$\hat{\mathcal{H}}_t = \text{LLM}_{\text{compact}}(\mathcal{H}_t \oplus p_c) \quad (1)$$

where p_c is a compaction instruction prompt. The agent blocks until the summary is generated, after which the compacted summary replaces the conversation history. The choice of threshold τ creates a fundamental tradeoff: lower thresholds keep context small but incur more blocking overhead and risk information loss, while higher thresholds preserve more context but allow latency to grow.

2.3. Motivating Measurements

Before introducing parallel compaction, we present motivating measurements that characterize the fundamental limitations of sequential compaction — why prompt engineering and input length alone cannot control summary volume, and how much wall-time overhead synchronous compaction imposes, motivating the need for a different approach.

2.3.1. Summary Output is Input-Invariant. We measured whether input length can control summary volume. Across four backbones on HotpotQA and LoCoMo, input length grows by 48× (2k→96k) while average output grows by only 3× (Table 1).

TABLE 1: Avg. summarization output vs. input length. Output stays nearly constant as input grows.

Input tokens	Avg output tokens	Output / Input
2,048	981	47.9 %
4,096	1,077	26.3 %
8,192	1,297	15.8 %
16,384	2,087	12.7 %
32,768	1,391	4.2 %
65,536	2,120	3.2 %
98,304	3,015	3.1 %

Takeaway #1: As input length grows 48× from 2k to 96k tokens, summarization output grows by only ∼3×: the model self-bounds its output regardless of how much context it receives.

2.3.2. Compaction Dominates Agent Wall Time. To quantify the cost of synchronous compaction, we run the agentic flow end-to-end under different context thresholds and measure how much of the total wall time is spent in compaction calls. Table 2 shows that synchronous compaction dominates wall time at low thresholds, consuming up to 62% of total execution on HotpotQA, and remains significant even at higher thresholds. The overhead compounds with frequency: at $\tau = 16$ k, compaction fires 15 times per run, each being a full stall, whereas at $\tau = 96$ k, only 2 compactions occur, but each processes a much longer context. Reducing per-call latency is therefore the primary lever for improving end-to-end throughput.

TABLE 2: Compaction fraction of E2E wall time vs. threshold.

Model	Threshold	# Compactions	Compact / E2E
gpt-oss-20B	16 k	15	51.3 %
	32 k	7	28.2 %
	64 k	3	14.5 %
	96 k	2	8.6 %
Llama-3.1-8B	16 k	15	62.4 %
	32 k	7	53.6 %
	64 k	3	24.4 %
	96 k	2	14.5 %

Takeaway #2: Synchronous compaction can become a bottleneck in long-horizon agentic flows, consuming a substantial fraction of end-to-end wall time.

3. Experimental Methodology

3.1. Benchmarks

We evaluate two agentic-flow benchmarks. **HotpotQA** [29] is a multi-hop QA dataset with Wikipedia-based question-answer pairs. **LoCoMo** [13] provides long multi-session conversations with multiple QAs each, using the multiple-choice question format [19]. We feed documents one at a time and ask each question immediately after it is read, so the conversation accumulates as a long-horizon agent's history would.

For each question, the backbone LLM generates a free-form answer graded by Qwen3-30B as an independent LLM-as-judge using a rubric template, run deterministically and blind to avoid self-preference bias. LLM evaluators tend to favor their own outputs [18, 27], so using a judge from a different model family than our backbones (GPT and Llama) ensures a fairer evaluation.

TABLE 3: Benchmark statistics.

Benchmark	Questions/doc	Doc length
HotpotQA	1	∼1.2k tokens
LoCoMo	6	∼0.7k tokens

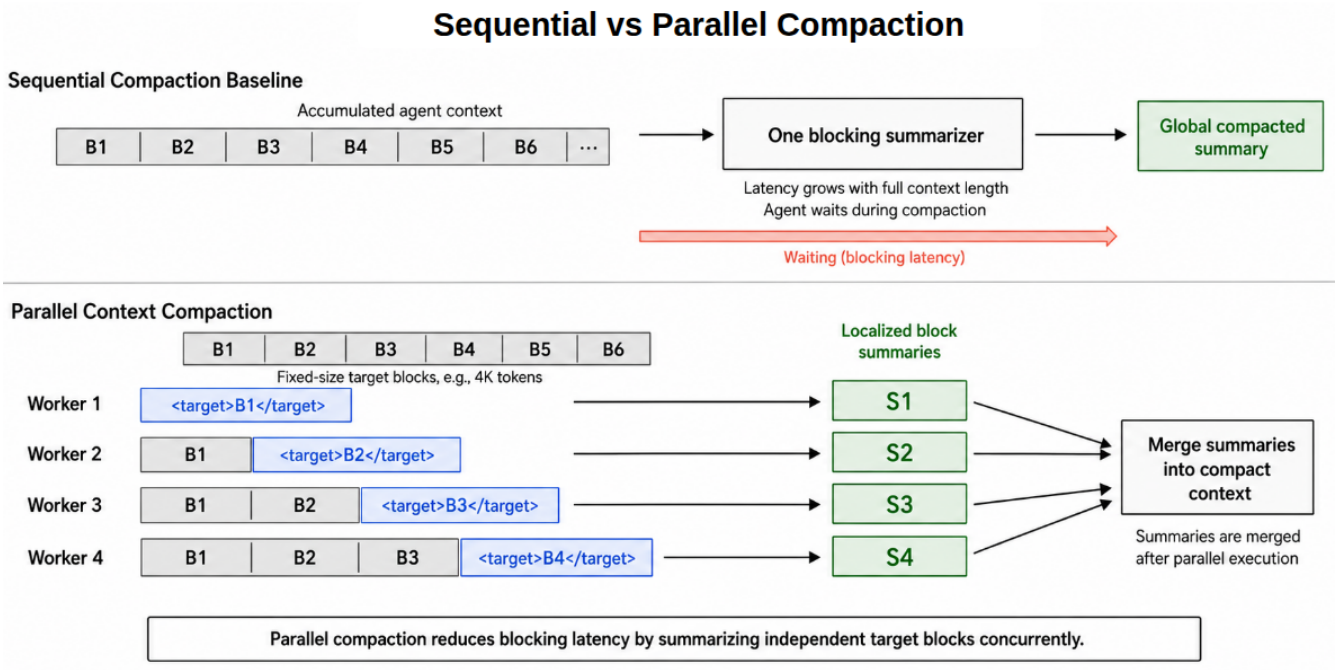


Figure 1: Sequential versus parallel compaction overview.

4. Parallel Compaction

When the conversation length exceeds a compaction threshold τ, parallel compaction proceeds through three phases (Figure 1):

1) **Snapshot & partition.** Copy the current conversation $\mathcal{X}$, record its length, and partition it into N contiguous blocks of size B tokens each, where $N = \lceil |\mathcal{X}|/B \rceil$ and B is a fixed configuration knob.
2) **Dispatch.** For each block $k \in \{1, \ldots, N\}$, build a prompt consisting of blocks 1 through k in order, with block k wrapped in a <TARGET_BLOCK> . . . </TARGET_BLOCK> marker. All N prompts are dispatched concurrently to the vLLM server.
3) **Merge.** Once all workers complete, the N per-block summaries are concatenated in block order to form the compacted history $\hat{\mathcal{H}}_t$, which replaces the original snapshot.

Prefix-aware target-at-end layout. Two natural alternatives exist: feed each worker only its block (*independent chunks*), or feed every worker the entire snapshot with a worker-specific marker (*shared full prefix*). Independent chunks break the prefix cache and lose cross-chunk context; shared full prefix breaks it too, since each worker's marker lands at a different offset. The prefix-aware target-at-end layout is the middle ground: the target block sits at the end of the visible prompt where attention is strongest, each worker's prefix strictly extends the previous worker's, preserving the cache, and cross-block context is maintained because each worker sees the full conversation history up to its block. This is a natural fit for agentic flows, where the conversation is built chronologically from left to right, so the prefix always reflects the causal order in which context was accumulated.

4.1. Models

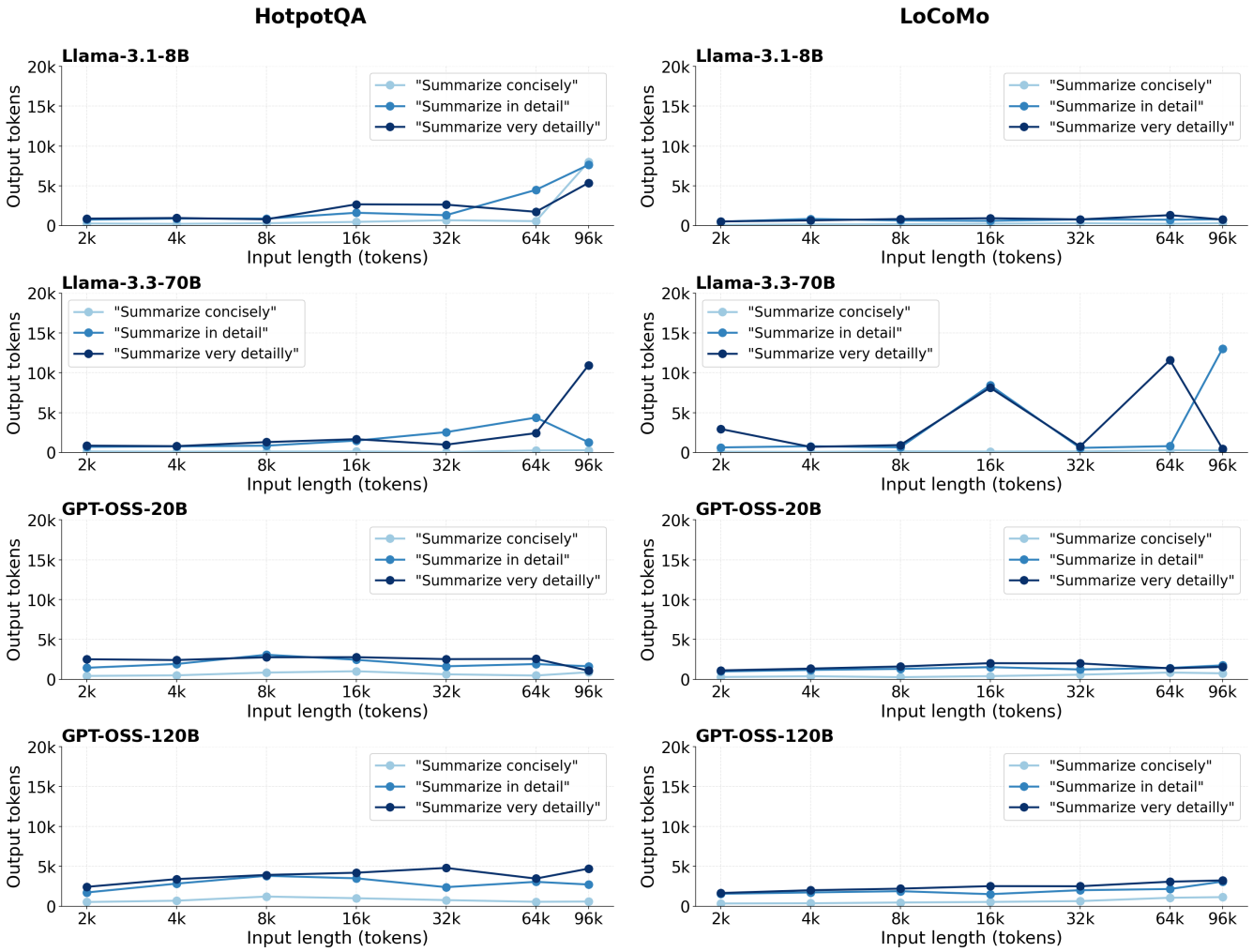
We evaluate four backbones to cover the regimes most relevant for production deployments: **Llama-3.1-8B** (small dense), **gpt-oss-20B** (medium MoE reasoning), **Llama-3.3-70B** (large dense), and **gpt-oss-120B** (large MoE reasoning). The four-backbone span captures small/large, dense/MoE, and reasoning/non-reasoning architectures. Table 4 lists each model's type, reasoning capability, context window, and GPU configuration. This selection also spans the practical cost spectrum, from a single-GPU deployment accessible to most practitioners to a large MoE model representative of frontier serving costs.

TABLE 4: Models used in our experiments.

Model	Type	Reasoning	Ctx	Hardware
Llama-3.1-8B	Dense	No	128k	1× H100
Llama-3.3-70B	Dense	No	128k	2× H100
gpt-oss-20B	MoE	Yes	128k	1× H100
gpt-oss-120B	MoE	Yes	128k	1× H100

4.2. Hardware and Serving Infrastructure

All experiments use vLLM with prefix caching and chunked prefill enabled. Llama-3.1-8B and gpt-oss-20B serve at TP=1 on a single H100; gpt-oss-120B serves at TP=1 with mxfp4 weights on a single H100; Llama-3.3-70B requires TP=2 across an H100 NVL pair. Each measurement runs on a dedicated GPU (or NVLink pair) with no other workloads sharing the device.



(a) HotpotQA (b) LoCoMo

Figure 2: Output tokens vs. input length for three prompt variants and four backbones under Sequential compaction.

5. Characterizing Sequential Compaction

Sequential compaction issues a single blocking summarization call over the entire conversation snapshot whenever the context exceeds the threshold τ. We characterize this baseline along two axes: output volume scaling, measuring how much the model produces as the input grows and as the prompt changes, and run-to-run stability, measuring how consistent that output is across repeated calls with the same input.

5.1. Prompt and Input Invariance

Figure 2 plots summarization output tokens against input length for all four backbones on HotpotQA (a) and LoCoMo (b), under three prompt variants. Two patterns dominate. First, all four backbones produce output tokens in a narrow range across the full 2 k to 96 k input sweep, which is far flatter than the 48× growth in input length would suggest. Second, switching the prompt from *concise* to *very detailed* barely moves the curves.

This behavior is consistent across dense and MoE architectures, as well as across reasoning and non-reasoning models on both benchmarks. Llama-3.3-70B tends to produce larger outputs than the other backbones at the same input length, but its behavior is non-deterministic across runs, making its output volume harder to predict. The operator has no effective prompt-side knob to grow summary volume as the conversation grows longer. The practical consequence is that sequential compaction progressively discards more and more of the conversation as context grows: at 2 k input, the model retains roughly 48% of the tokens in the summary, but at 96 k, that figure drops to ∼3%.

Takeaway #3: Changing the prompt has no significant impact on output length: models largely ignore length instructions and self-bound their output regardless of whether the prompt asks for a concise or very detailed summary. The operator cannot use prompt engineering to control compaction volume.

5.2. Run-to-Run Instability

Beyond input invariance, both output length and content vary substantially from run to run. This is a fundamental but under-characterized property of LLM-based compaction: given an identical conversation snapshot and the same prompt, the model produces a different summary on every call. The output length may shift by hundreds of tokens, and the semantic content of the summary changes accordingly. This matters because the compacted history directly conditions all subsequent reasoning steps—an agent that receives a shorter or differently-framed summary on one run will follow a different reasoning path than on another, even though the underlying task is identical. Instability worsens with context length: longer inputs give the model more room to vary which facts it selects, how it phrases them, and how much it elaborates, amplifying the token-count and semantic spread across runs. To systematically characterize this behavior, we sweep input length from 4 k to 96 k tokens and, at each point, run compaction ten times under identical prompt and input conditions with temperature-driven sampling, measuring how much the output length varies and how similar the generated summaries are to one another.

Output length variability: coefficient of variation. To quantify how much the output token count shifts across runs, we use the coefficient of variation (CV), defined as the ratio of the standard deviation to the mean of the output length across the ten repeated compaction calls. A CV of 0% indicates perfectly consistent output length; a CV of 100% means the standard deviation equals the mean, reflecting a very wide spread relative to the average output size. We prefer CV over raw standard deviation because it is scale-independent — a CV of 30% means the same relative spread whether the model produces 100 or 5,000 tokens on average, making it comparable across models and input lengths. In the compaction context, high CV means that the operator cannot predict how many tokens will be consumed by the summary on any given run, which makes memory budgeting and context threshold planning unreliable.

Summary content variability: cosine similarity. Token count alone does not capture whether two summaries actually say the same thing — a model could produce outputs of identical length while selecting entirely different facts. To measure semantic consistency, we embed each of the ten summaries using a fixed sentence encoder and compute the average pairwise cosine similarity across all pairs. A cosine similarity of 1.0 means every run produces semantically identical text; values near 0 indicate that runs share almost no semantic content. Low cosine similarity is particularly damaging in agentic flows: if the compacted summary omits a key fact in one run but retains it in another, downstream QA accuracy will vary between runs not because the model reasoned differently, but because it received a different context—making performance non-reproducible even on a fixed benchmark.

TABLE 5: Stability across input sizes (4–96 k).

Model	Input	Coefficient of Variation (%) ↓		Cosine Similarity ↑	
		Value	Δ	Value	Δ
gpt-oss-20B	4 k	24.5%	–	0.777	–
	8 k	21.5%	−3.0	0.741	−0.036
	16 k	38.7%	+14.2	0.726	−0.051
	32 k	27.3%	+2.8	0.654	−0.123
	64 k	42.0%	+17.5	0.713	−0.064
	96 k	42.9%	+18.4	0.624	−0.153
gpt-oss-120B	4 k	19.8%	–	0.825	–
	8 k	36.0%	+16.2	0.857	+0.032
	16 k	16.2%	−3.6	0.783	−0.042
	32 k	70.8%	+51.0	0.751	−0.074
	64 k	62.0%	+42.2	0.760	−0.065
	96 k	68.6%	+48.8	0.619	−0.206
Llama-3.1-8B	4 k	34.1%	–	0.652	–
	8 k	33.0%	−1.1	0.608	−0.044
	16 k	26.4%	−7.7	0.576	−0.076
	32 k	48.5%	+14.4	0.665	+0.013
	64 k	54.9%	+20.8	0.550	−0.102
	96 k	84.5%	+50.4	0.472	−0.180
Llama-3.3-70B	4 k	24.4%	–	0.729	–
	8 k	41.3%	+16.9	0.620	−0.109
	16 k	35.4%	+11.0	0.590	−0.139
	32 k	47.7%	+23.3	0.630	−0.099
	64 k	26.4%	+2.0	0.579	−0.150
	96 k	171.6%	+147.2	0.491	−0.238

Block size as a stability driver. The CV and cosine similarity measurements above serve a concrete selection purpose: identifying which block size yields compaction outputs that are stable enough to be reliable in production. Smaller block sizes engage more workers, each summarizing a shorter, more focused segment, which reduces the model's degrees of freedom and produces more consistent outputs from run to run. Larger blocks force the model to compress more content in a single call, increasing the variance in what it chooses to retain and how it phrases it. Block size is therefore not only a throughput lever but also a stability driver.

Across all models in Table 5, the 4 k input point is the most favorable: CV ranges from 19.8% to 34.1% and cosine similarity stays between 0.65 and 0.83 across all backbones. Some models show slightly better cosine similarity or lower CV at other input sizes, though these gains are not consistent across backbones. In general, as input grows beyond 4 k, both metrics tend to degrade with no sustained recovery at larger sizes. This makes 4 k a natural operating point: short enough to limit drift, yet large enough to capture meaningful context. In the parallel compaction setting, a 4 k block size means each worker's target segment is 4 k tokens, wrapped in an XML marker, so each individual summarization call operates at the input length where models are most stable.

Takeaway #4: Run-to-run instability is intrinsic to LLM-based compaction and compounds as context grows — both output length and summary content become increasingly unpredictable.

Having established that instability grows with input length, we now fix the input at 4 k — the most stable point identified in Table 5 — and examine how models respond to different prompt-length instructions. We vary the prompt across three levels of requested detail: summarize with one sentence, one paragraph, and three paragraphs. The goal is to assess whether longer prompts can reduce output variability or whether instability persists regardless of the prompt. Figure 3 shows output token counts across ten repeated runs per model and prompt variant.

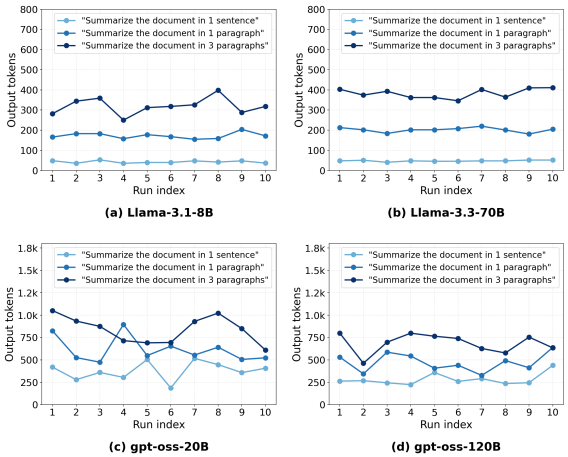


Figure 3: Output tokens per run across 10 repeated runs at a fixed 4 k input for three prompt-length variants (light to dark = shorter to longer requested summary length).

Llama instruct models are noticeably more consistent across runs than gpt-oss models, with tighter scatter within each prompt variant. All four backbones scale output tokens from roughly 400 to 600 tokens as the prompt requests more detail, showing that models do respond to prompt-length instructions at this input size. The narrow output range across all models and prompts suggests that models have a strong prior on summary length.

Why LLMs produce short summaries. A 500-token summary corresponds roughly to half a page to one page of dense text, a length that aligns well with summaries found in standard NLP summarization corpora. Many models are trained or instruction-tuned on document — summary datasets where reference summaries are often in this range, so they may reproduce this learned length prior at inference time, regardless of how long the source document is. This provides one explanation for why neither input length nor prompt instructions significantly move the output: the model tends to revert to a summary length internalized from training data. As a result, a 500-token summary at 96 k input retains less than 1% of the original context. [20]

Takeaway #5: Summarization output is bounded by the model's internalized prior on summary length: during training, models absorb not just what a summary should contain but also how long it should be, anchoring output length to the distribution of reference summaries in training data regardless of how much input context is provided.

Table 6 quantifies run-to-run stability across the three prompt-length variants at a fixed 4 k input. CV and cosine similarity values are largely consistent across variants for each model, with no backbone showing a large or monotonic improvement as the prompt becomes more detailed. Llama-3.3-70B has the lowest CV across all variants, consistent with the tighter scatter in Figure 3.

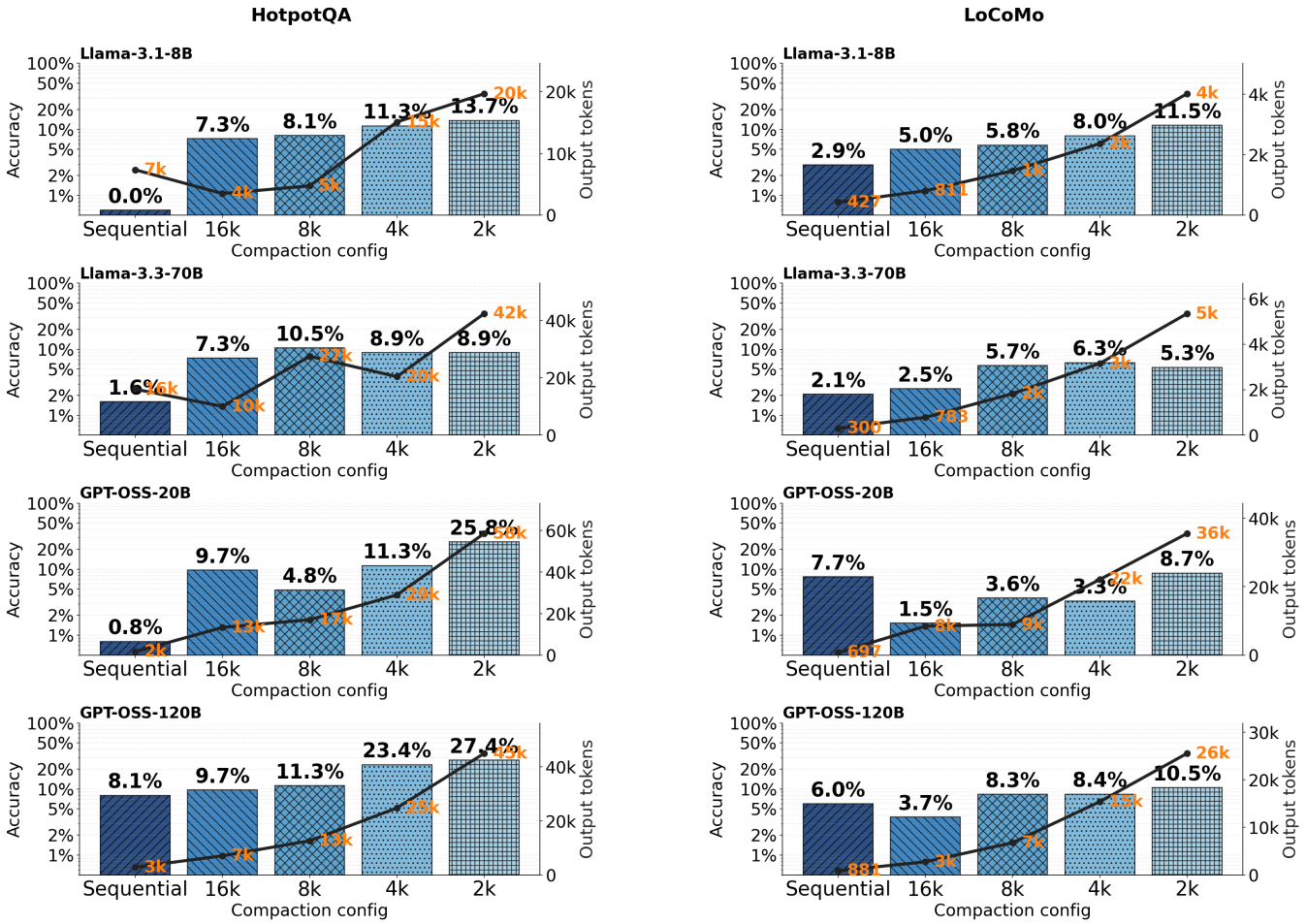
TABLE 6: Stability across prompt-length variants at fixed 4 k input.

Model	Prompt	CV (%) ↓	Cosine ↑
gpt-oss-20B	Summarize with 1 sentence	27.2%	0.704
	Summarize with 1 paragraph	23.0%	0.747
	Summarize with 3 paragraphs	18.1%	0.788
gpt-oss-120B	Summarize with 1 sentence	23.5%	0.732
	Summarize with 1 paragraph	21.8%	0.819
	Summarize with 3 paragraphs	15.9%	0.820
Llama-3.1-8B	Summarize with 1 sentence	14.3%	0.815
	Summarize with 1 paragraph	8.6%	0.719
	Summarize with 3 paragraphs	13.1%	0.718
Llama-3.3-70B	Summarize with 1 sentence	6.9%	0.922
	Summarize with 1 paragraph	5.9%	0.799
	Summarize with 3 paragraphs	6.2%	0.831

Long-horizon compaction as long document summarization. The structure of a long-horizon agentic conversation closely mirrors that of a long document: it is built chronologically, turn by turn, from left to right, with each exchange adding context sequentially. This framing motivates treating compaction as a long document summarization problem, where divide-and-conquer approaches split the input into chunks and summarize each independently. Our parallel compaction design follows this intuition but adds a critical constraint: each block worker receives the full conversation history up to its block rather than just the block itself, preserving cross-block context dependencies and enabling prefix cache reuse across workers — properties that a naive chunking approach would not preserve.

Role of the XML marker. In the prefix-aware layout, each worker receives the full conversation history up to its block, which raises an ambiguity: without explicit delimiters, the model would attempt to summarize the entire prefix rather than its assigned block. The XML marker resolves this by clearly delineating the target region at the end of the prompt, where attention is strongest. This allows the model to attend to the full preceding context for cross-block coherence while focusing its summarization output on the marked segment only.

Threshold and block size interaction. The compaction threshold τ and block size B together determine how many workers fire per compaction event: $N = \lceil \tau/B \rceil$. At a fixed threshold of $\tau = 96$ k tokens, reducing the block size from 16 k to 4 k increases the number of parallel workers from 6 to 24. The operator therefore has two independent knobs: τ controls how often compaction fires, and B controls how many workers run and how much output is produced per event.



(a) HotpotQA (b) LoCoMo

Figure 4: Accuracy per model and configuration on (a) HotpotQA and (b) LoCoMo.

6. Characterizing Parallel Compaction

Figure 4 reports accuracy and compaction output token count per configuration on HotpotQA and LoCoMo, with Sequential and each block size on the x-axis, accuracy as histogram bars, and output tokens overlaid. For each configuration, we feed a 100 k-token context to the model, ask it to summarize, and use Qwen3-30B as a judge to assess whether the summary preserves the information needed to answer the corresponding questions. The 16 k block size produces the fewest output tokens; as block size decreases, more workers run in parallel, total output tokens increase, and more of the original context is preserved in the summary. Accuracy improves over the Sequential baseline as block size decreases, and this relationship is approximately linear — as total compaction output grows with the number of workers, accuracy increases monotonically across the block-size sweep, consistently across all configurations. It is worth noting that accuracy does not reach 100% even at the smallest block size, as performance is also bounded by the model's own reasoning capability on the downstream task, not solely by information preservation in the summary.

Takeaway #6: Block size directly controls how much context survives compaction, improving accuracy over sequential by preserving more context in the summary.

Having established that parallel compaction improves accuracy over sequential, we now evaluate its system-level performance in a multi-turn agentic flow. The agent processes a sequence of QA tasks, accumulates context across turns, and triggers compaction at a fixed threshold of $\tau = 96$ k tokens. Table 7 reports end-to-end wall time, throughput, compaction decode tokens, QA decode tokens, and average compaction throughput. Compaction decode tokens measure how many tokens the compaction workers generate, while QA decode tokens capture the agent's downstream answer generation. Average compaction throughput accounts for both prefill and decode cost of the compaction.

Table 7 shows end-to-end speedups, though measuring a fair throughput comparison is non-trivial: compaction output token counts fluctuate across runs and LLMs cannot reliably be constrained to a specific length, making byte-identical token counts across configurations difficult to achieve.

TABLE 7: **HotpotQA: Sequential vs Parallel across block sizes.** Fixed threshold τ = 96k. Sequential is the single-call blocking compaction baseline; Parallel uses one worker per block (block size sweep: 16k, 8k, 4k, 2k tokens; smaller block $\Rightarrow$ more workers). The Δ column shows the speedup ratio over Sequential. Green = improvement, red = regression.

Model	Variant	Block size	E2E Wall (s) ↓		E2E Throughput (tok/s) ↑		Compaction Decode (tok)		QA Decode (tok)		Compaction Throughput (ms/tok) ↓	
			Value	Δ	Value	Δ	Value	Δ	Value	Δ	Value	Δ
HotpotQA at τ = 96k												
gpt-oss-20B	Sequential	–	86.9	1.00×	145.2	1.00×	1,493	1.00×	11,127	1.00×	5.34	1.00×
	Parallel	16k	100.7	0.86×	147.1	1.01×	3,098	2.07×	11,704	1.05×	5.59	0.95×
	Parallel	8k	101.8	0.85×	168.1	1.16×	5,401	3.62×	11,710	1.05×	3.39	1.58×
	Parallel	4k	105.1	0.83×	205.3	1.41×	10,017	6.71×	11,554	1.04×	2.12	2.52×
	Parallel	2k	108.3	0.80×	253.8	1.75×	16,180	10.84×	11,303	1.02×	1.53	3.48×
gpt-oss-120B	Sequential	–	214.5	1.00×	51.0	1.00×	1,462	1.00×	9,480	1.00×	7.28	1.00×
	Parallel	16k	276.0	0.78×	49.8	0.98×	4,116	2.81×	9,615	1.01×	7.64	0.95×
	Parallel	8k	270.3	0.79×	59.7	1.17×	6,994	4.78×	9,136	0.96×	5.22	1.39×
	Parallel	4k	217.7	0.99×	90.0	1.76×	9,894	6.77×	9,701	1.02×	3.58	2.04×
	Parallel	2k	233.1	0.92×	109.0	2.14×	16,486	11.28×	8,913	0.94×	2.51	2.91×
Llama-3.1-8B	Sequential	–	150.8	1.00×	45.5	1.00×	1,776	1.00×	5,082	1.00×	14.31	1.00×
	Parallel	16k	131.7	1.14×	45.2	0.99×	681	0.38×	5,272	1.04×	32.96	0.43×
	Parallel	8k	104.6	1.44×	62.8	1.38×	1,151	0.65×	5,424	1.07×	17.89	0.80×
	Parallel	4k	105.8	1.43×	71.0	1.56×	2,199	1.24×	5,313	1.05×	10.43	1.37×
	Parallel	2k	119.4	1.26×	83.8	1.84×	4,575	2.58×	5,433	1.07×	6.57	2.18×
Llama-3.3-70B	Sequential	–	373.5	1.00×	39.0	1.00×	8,582	1.00×	5,994	1.00×	22.62	1.00×
	Parallel	16k	289.7	1.29×	39.6	1.01×	5,317	0.62×	6,145	1.03×	19.79	1.14×
	Parallel	8k	268.5	1.39×	46.4	1.19×	6,823	0.79×	5,647	0.94×	13.29	1.70×
	Parallel	4k	266.6	1.40×	52.5	1.35×	8,360	0.97×	5,645	0.94×	10.62	2.13×
	Parallel	2k	292.4	1.28×	64.1	1.64×	12,303	1.43×	6,427	1.07×	8.09	2.80×

TABLE 8: **LoCoMo: Sequential vs Parallel across block sizes.** Same protocol as Table 7: fixed threshold τ = 96k, Parallel variants sweeping block size from 16k down to 2k tokens. Δ columns report the speedup over Sequential for the corresponding metric (green improvement, red regression).

Model	Variant	Block size	E2E Wall (s) ↓		E2E Throughput (tok/s) ↑		Compaction Decode (tok)		QA Decode (tok)		Compaction Throughput (ms/tok) ↓	
			Value	Δ	Value	Δ	Value	Δ	Value	Δ	Value	Δ
LoCoMo at τ = 96k												
gpt-oss-20B	Sequential	–	991.4	1.00×	134.0	1.00×	6,344	1.00×	126,533	1.00×	5.23	1.00×
	Parallel	16k	925.6	1.07×	138.2	1.03×	2,909	0.46×	125,031	0.99×	5.57	0.94×
	Parallel	8k	1001.2	0.99×	140.1	1.05×	5,860	0.92×	134,431	1.06×	3.51	1.49×
	Parallel	4k	985.0	1.01×	145.0	1.08×	8,368	1.32×	134,414	1.06×	2.42	2.16×
	Parallel	2k	1038.0	0.96×	143.7	1.07×	16,435	2.59×	132,750	1.05×	1.56	3.36×
gpt-oss-120B	Sequential	–	4905.0	1.00×	19.9	1.00×	1,479	1.00×	96,196	1.00×	7.52	1.00×
	Parallel	16k	3297.0	1.49×	30.2	1.52×	1,487	1.01×	98,081	1.02×	19.37	0.39×
	Parallel	8k	3065.3	1.60×	34.4	1.73×	3,769	2.55×	101,795	1.06×	7.42	1.01×
	Parallel	4k	3601.6	1.36×	30.7	1.54×	8,308	5.62×	102,150	1.06×	7.03	1.07×
	Parallel	2k	3851.9	1.27×	32.1	1.61×	17,816	12.05×	105,808	1.10×	4.49	1.68×
Llama-3.1-8B	Sequential	–	495.2	1.00×	52.4	1.00×	542	1.00×	25,406	1.00×	12.80	1.00×
	Parallel	16k	555.9	0.89×	55.5	1.06×	616	1.14×	30,212	1.19×	32.82	0.39×
	Parallel	8k	530.1	0.93×	53.2	1.02×	1,198	2.21×	27,010	1.06×	18.28	0.70×
	Parallel	4k	594.6	0.83×	60.1	1.15×	2,245	4.14×	33,469	1.32×	11.06	1.16×
	Parallel	2k	607.0	0.82×	63.4	1.21×	4,357	8.04×	34,110	1.34×	6.81	1.88×
Llama-3.3-70B	Sequential	–	1194.5	1.00×	38.9	1.00×	1,451	1.00×	45,043	1.00×	22.32	1.00×
	Parallel	16k	1495.6	0.80×	41.5	1.07×	6,708	4.62×	55,359	1.23×	22.12	1.01×
	Parallel	8k	1306.8	0.91×	40.0	1.03×	4,179	2.88×	48,050	1.07×	17.58	1.27×
	Parallel	4k	1275.2	0.94×	42.1	1.08×	8,494	5.85×	45,193	1.00×	10.27	2.17×
	Parallel	2k	1388.7	0.86×	50.9	1.31×	23,009	15.86×	47,655	1.06×	6.81	3.28×

Table 9 therefore selects runs from the full sweep where Sequential and Parallel produce comparable compaction decode volumes, isolating the throughput improvement attributable to parallelism rather than token count differences.

TABLE 9: Parallel achieves lower TPOT at matched decode volume.

#	Model	Benchmark	Block	Seq. (tok)	Par. (tok)	Δ Throughput
1	Llama-3.3-70B	HotpotQA	4 k	8,582	8,360	2.13×
2	Llama-3.3-70B	HotpotQA	8 k	8,582	6,823	1.70×
3	gpt-oss-20B	LoCoMo	8 k	6,344	5,860	1.49×
4	Llama-3.1-8B	HotpotQA	4 k	1,776	2,199	1.37×

Prefill cost limits parallelism. Average compaction throughput measures the combined prefill and decode latency per output token during compaction. In the parallel setting, Workers share the common conversation prefix via the prefix cache, but each worker must additionally prefill its own unique block content, which is not cached. At the 16 k block size, this uncached portion is 16 k tokens per worker, incurring substantial prefill overhead that the parallelism gain cannot compensate for, resulting in consistently worse throughput compared to smaller block sizes.

Prompt sensitivity under parallel compaction. While sequential compaction largely ignores prompt length instructions, parallel compaction restores some prompt sensitivity. Figure 5 shows gpt-oss-120B output tokens across three prompt variants and all block sizes on HotpotQA and LoCoMo; other backbones exhibit similar trends. The model does respond to the prompt within each block size, with more detailed prompts producing more tokens. Combined with block size, this gives the operator two independent levers: block size controls the coarse volume of compaction output, while the prompt can further tune output length within each block.

TABLE 10: gpt-oss-120B compaction output / 96 k input ratio (%).

Config	HotpotQA			LoCoMo		
	Concise	Detailly	Very Detailly	Concise	Detailly	Very Detailly
Sequential	0.79%	2.98%	4.16%	0.92%	3.74%	4.05%
16k	3.62%	7.35%	13.87%	2.87%	7.32%	7.05%
8k	8.48%	13.14%	15.07%	7.05%	12.47%	14.22%
4k	12.37%	25.75%	34.13%	16.02%	24.32%	26.57%
2k	28.16%	46.73%	50.98%	26.59%	42.44%	47.34%

As shown in Table 10, this two-axis control is quantitatively meaningful. By tuning the block size alone, the operator can scale compaction output to roughly 25% of the original context, and by additionally selecting a more detailed prompt, the output can reach up to 50%, offering fine-grained, predictable control over how much context survives each compaction call.

Output token tradeoff. While the results above demonstrate that output volume can scale up to 50% of the original context, retaining that much is not the goal in itself. The operator can equally scale down by choosing a concise

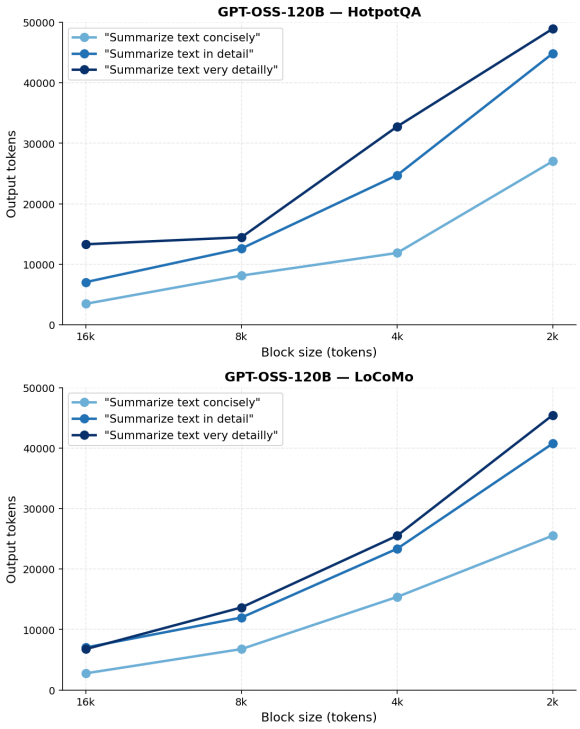


Figure 5: GPT-OSS-120B output tokens vs. block size across three prompt variants on HotpotQA (left) and LoCoMo (right).

prompt, reducing output significantly. The key insight is bidirectional control: block size and prompt together allow the operator to tune compaction volume precisely in either direction. Keeping more tokens in the summary is not always better: longer summaries re-introduce the same attention cost and context rot effects that motivated compaction in the first place. As the retained context grows, the model must again attend over a longer, lower-signal history, degrading reasoning quality before the next compaction trigger. The optimal operating point is therefore not the maximum but the minimum sufficient to preserve task-relevant information, and the two-axis control allows the operator to target that point precisely.

Takeaway #7: Whether the technique is compaction, token-efficient tool design, or just-in-time retrieval, recent practitioner guidance converges on a common principle: keep inside the context window only the smallest set of high-signal tokens likely to maximize the desired outcome [3].

7. Related Work

Context management. Managing the growing context of long-horizon LLM agents has been studied along several axes [9, 11]. Existing approaches range from simple truncation to summarization-based compaction and complex retrieval-based memory systems. It has been shown that truncation-based approaches are insufficient for maintaining coherence in long conversations, as summarization methods

significantly outperform sliding window baselines [26]. Token-level compression methods reduce context length by removing low-information tokens, but these approaches lack semantic awareness and disrupt coherence in multi-step reasoning chains [7, 8]. Retrieval-based approaches such as MemGPT store conversation history in external memory and retrieve relevant segments on demand, but require dedicated retrieval infrastructure and add latency for each lookup [17].

Latent space compression. KV-cache and latent-space compression methods offer complementary approaches: Cartridges [5] pre-train lightweight KV-cache representations offline to reduce inference cost on large corpora, while Zweiger et al. [34] compress KV caches at runtime via attention-pattern matching. However, Cartridges require offline training and are not applicable at runtime, while KV-cache compression operates in the latent space, making the compressed representation opaque to humans, non-transferable across models, and dependent on white-box access to the model's internal cache. LLM-based summarization, by contrast, produces human-readable text that is model-agnostic and requires no access to model internals.

Effective context length. Beyond keeping the conversation within the model's hard context limit, recent work suggests that keeping fewer, higher-signal tokens inside the window yields better downstream accuracy, motivating more frequent compression of the active context. One study [14] shows that across 13 models claiming 128k to 2M token windows, the effective context length (the length at which accuracy stays within 85% of the short-context baseline) is only between roughly 1k and 8k tokens. Another study [4] sharpens this by controlling for retrieval: performance still degrades substantially with input length even when the model retrieves all evidence with exact match, and even when non-evidence tokens are attention-masked, establishing input length itself as a first-order cause of reasoning degradation. These results reframe compaction not just as a mechanism to fit within the context window, but as a primary lever for keeping the active context inside the regime where attention functions effectively.

Parallel document summarization. Divide-and-conquer approaches for long document summarization split the input into chunks, summarize each independently, and merge the results [16, 24, 32, 33]. Recent theoretical work characterizes when such chunked processing outperforms single-pass approaches [28]. Our work differs in targeting runtime context compaction in agentic flows, parallelizing summarization via prefix-aware blocks where each block sees the full conversation history preceding it, preserving cross-block context dependencies, with only a small additional cost from placing a target marker at the end of each prefix, while enabling prefix cache reuse across workers.

LLMs ignore length instructions. Multiple benchmarks have shown that LLMs systematically fail to follow explicit length constraints: models struggle to match specified word or token counts, especially at longer targets [1, 25, 30], and performance degrades further in long-context scenarios [21]. LLMs also exhibit verbosity compensation, generating excessively long outputs when uncertain [31], and lack the ability to track their own output length during generation [23]. Evaluation of long-form generation confirms these limitations persist even when models are explicitly instructed [22]. These findings support our design choice: since prompt instructions cannot reliably control summary volume, we use the number of parallel workers as the primary control knob for compaction output. Unlike prior work, we target runtime agentic conversation compaction with no offline training, retrieval infrastructure, or white-box model access, and introduce a prefix-aware target-at-end design that preserves cross-block context and enables prefix cache reuse.

8. Discussion and Future Work

Dynamic block size and prompt selection. In this work, we use a fixed block size and a single compaction prompt. A natural extension is to adapt both per block based on content type: tool call outputs with shallow structure could use larger blocks and a concise prompt, while math-heavy or insight-dense segments could use smaller blocks with a detailed prompt, steering compaction granularity and verbosity where it matters most.

Model fine-tuning for better marker awareness. Our prefix-aware layout inserts XML markers to delimit block boundaries. Current models were not trained to attend to these markers during summarization. Fine-tuning on marker-annotated compaction traces could improve boundary awareness and produce more faithful per-block summaries.

Asynchronous compaction. The current design blocks the agent until all workers finish. An async variant would run block workers in the background while the main thread continues serving inference on the residual context, merging the completed summaries once the current turn finishes. Tool calls offer a natural trigger: whenever the agent is waiting on `pytest`, a shell command, or any external API, the GPU is idle, and compaction can proceed at zero opportunity cost.

9. Conclusion

In this work, we characterized sequential and parallel context compaction for long-horizon LLM agents across four backbones and two benchmarks. Our detailed characterization reveals that summarization output is largely input- and prompt-invariant, making prompt engineering an unreliable control knob for summary volume. We introduced parallel block compaction, which gives the operator direct control over summary volume through block count and increases compaction throughput even at matched decode volume.

References

[1] A. Akinfaderin, S. Subramanian, and A. Sehwag, "Plan-and-write: Structure-guided length control for llms without model retraining," *arXiv preprint arXiv:2511.01807*, 2025.

[2] Anthropic, "Claude code: Best practices for agentic coding," 2025. [Online]. Available: https://platform.claude.com/docs/en/build-with-claude/compaction

[3] ——, "Effective context engineering for ai agents," https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents, 2025, accessed: 2026-05-11.

[4] Y. Du, M. Tian, S. Ronanki, S. Rongali, S. Bodapati, A. Galstyan, A. Wells, R. Schwartz, E. A. Huerta, and H. Peng, "Context length alone hurts llm performance despite perfect retrieval," *arXiv preprint arXiv:2510.05381*, 2025.

[5] S. Eyuboglu, R. Ehrlich, S. Arora, N. Guha, D. Zinsley, E. Liu, W. Tennien, A. Rudra, J. Zou, A. Mirhoseini *et al.*, "Cartridges: Lightweight and general-purpose long context representations via self-study," *arXiv preprint arXiv:2506.06266*, 2025.

[6] K. Hong, A. Troynikov, and J. Huber, "Context rot: How increasing input tokens impacts llm performance," Chroma, Tech. Rep., July 2025. [Online]. Available: https://research.trychroma.com/context-rot

[7] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "Llmlingua: Compressing prompts for accelerated inference of large language models," in *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023, pp. 13 358–13 376.

[8] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 1658–1677.

[9] M. Kang, W.-N. Chen, D. Han, H. A. Inan, L. Wutschitz, Y. Chen, R. Sim, and S. Rajmohan, "Acon: Optimizing context compression for long-horizon llm agents," *arXiv preprint arXiv:2510.00615*, 2025.

[10] LangChain, "Langchain: Building applications with llms through composability," 2023. [Online]. Available: https://github.com/langchain-ai/langchain

[11] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the association for computational linguistics*, vol. 12, pp. 157–173, 2024.

[12] LlamaIndex, "Llamaindex: A data framework for llm applications," 2023. [Online]. Available: https://github.com/run-llama/llama_index

[13] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Fang, "Evaluating very long-term conversational memory of llm agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 13 851–13 870.

[14] A. Modarressi, H. Deilamsalehy, F. Dernoncourt, T. Bui, R. A. Rossi, S. Yoon, and H. Schütze, "Nolima: Long-context evaluation beyond literal matching," *arXiv preprint arXiv:2502.05167*, 2025.

[15] OpenAI, "Codex prompting guide," 2025, [Online]. Available: https://developers.openai.com/cookbook/examples/gpt-5/codex_prompting_guide.

[16] L. Ou and M. Lapata, "Context-aware hierarchical merging for long document summarization," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 5534–5561.

[17] C. Packer, V. Fang, S. Patil, K. Lin, S. Wooders, and J. Gonzalez, "Memgpt: towards llms as operating systems." 2023.

[18] A. Panickssery, S. R. Bowman, and S. Feng, "Llm evaluators recognize and favor their own generations," *Advances in Neural Information Processing Systems*, vol. 37, pp. 68 772–68 802, 2024.

[19] Percena, "LoCoMo-MC10: A 10-choice multiple-choice version of locomo," https://huggingface.co/datasets/Percena/locomo-mc10, 2026, hugging Face dataset. Accessed: 2026-05-19.

[20] M. Ravaut, A. Sun, N. Chen, and S. Joty, "On context utilization in summarization with large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 2764–2781.

[21] X. Wu, M. Wang, Y. Liu, X. Shi, H. Yan, L. Xiangju, J. Zhu, and W. Zhang, "Lifbench: Evaluating the instruction following performance and stability of large language models in long-context scenarios," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 16 445–16 468.

[22] Y. Wu, M. S. Hee, Z. Hu, and R. K.-W. Lee, "Longgenbench: Benchmarking long-form generation in long context llms," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 6851–6872.

[23] M. Xiao, A. Wang, Q. Hu, Z. Miao, H. Shen, L. Wang, W. Luo, and J. Su, "Can llms track their output length? a dynamic feedback mechanism for precise length regulation," *arXiv preprint arXiv:2601.01768*, 2026.

[24] S. Xiao, Z. Lin, W. Gao, H. Chen, and Y. Zhang, "Long context scaling: Divide and conquer via multi-agent question-driven collaboration," *arXiv preprint arXiv:2505.20625*, 2025.

[25] J. Xie and H.-y. Lee, "Prompt-based one-shot exact length-controlled generation with llms," *arXiv preprint arXiv:2508.13805*, 2025.

[26] J. Xu, A. Szlam, and J. Weston, "Beyond goldfish memory: Long-term open-domain conversation," in *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: long papers)*, 2022, pp. 5180–5197.

[27] W. Xu, G. Zhu, X. Zhao, L. Pan, L. Li, and W. Wang, "Pride and prejudice: Llm amplifies self-bias in self-refinement," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 15 474–15 492.

[28] Z. Xu, S. Zhu, J. Wang, J. Wang, B. Athiwaratkun, C. Wang, J. Zou, and C. Zhang, "When does divide and conquer work for long context llm? a noise decomposition framework," *arXiv preprint arXiv:2506.16411*, 2025.

[29] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "Hotpotqa: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 conference on empirical methods in natural language processing*, 2018, pp. 2369–2380.

[30] W. Zhang, Z. Zhou, K. Wang, J. Fang, R. Xu, Y. Zhang, R. Wang, G. Zhang, X. Li, L. Sun *et al.*, "Lifebench: Evaluating length instruction following in large language models," *Advances in Neural Information Processing Systems*, vol. 38, 2026.

[31] Y. Zhang, S. S. S. Das, and R. Zhang, "Demystify verbosity compensation behavior of large language models," in *Proceedings of the 2nd Workshop on Uncertainty-Aware NLP (UncertaiNLP 2025)*, 2025, pp. 160–178.

[32] Y. Zhang, R. Sun, Y. Chen, T. Pfister, R. Zhang, and S. Ö. Arık, "Chain of agents: Large language models collaborating on long-context tasks," *Advances in Neural Information Processing Systems*, vol. 37, pp. 132 208–132 237, 2024.

[33] Z. Zhou, C. Li, X. Chen, S. Wang, Y. Chao, Z. Li, H. Wang, R. An, Q. Shi, Z. Tan, X. Han, X. Shi, Z. Liu, and M. Sun, "Llm×mapreduce: Simplified long-sequence processing using large language models," 2024. [Online]. Available: https://arxiv.org/abs/2410.09342

[34] A. Zweiger, X. Fu, H. Guo, and Y. Kim, "Fast kv compaction via attention matching," *arXiv preprint arXiv:2602.16284*, 2026.